

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**

Bibliotecas: Pandas, NumPy, SciPy, Matplotlib e Seaborn

NumPy - acesso, inspeção e índice

Aula 3

Código da aula: [DADOS]ANO1C2B3S20A3

**Bibliotecas: Pandas,
NumPy, SciPy,
Matplotlib e Seaborn**

Mapa da Unidade 5 Componente 3

NumPy: manipulação
de array.

semana
22

Pandas: manipulação
de datas.

semana
27

semana
19
SciPy.

semana
20

Você está aqui!
NumPy: acesso,
inspeção e índice.

semana
29

Matplotlib: estrutura.

**Bibliotecas: Pandas,
NumPy, SciPy,
Matplotlib e Seaborn**

Mapa da Unidade 5 Componente 3

Você está aqui!

NumPy - acesso, inspeção e índice

Aula 3

Código da aula:
[DADOS]ANO1C2B3S20A3

20



Objetivos da aula

- Conhecer a relação entre matrizes, vetores e a biblioteca NumPy;
- Criar e inspecionar um array.



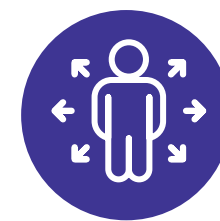
Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou internet;
- Software Anaconda/Jupyter Notebook instalado ou similar.



Duração da aula

50 minutos.



Competências técnicas

- Ser proficiente em linguagens de programação para manipular e analisar grandes conjuntos de dados;
- Usar técnicas para explorar e analisar dados, aplicar modelos estatísticos, identificar padrões, realizar inferências e tomar decisões baseadas em evidências.



Competências socioemocionais

- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados;
- Trabalhar em equipes multifuncionais colaborando com colegas, gestores e clientes.

Construindo o conceito

Matrizes e NumPy

A matriz pode ser representada no NumPy por meio da função **array**. Observe que, na primeira linha da matriz, há $[3, 1]$ e, na segunda linha, $[2, 4]$. Ambos dentro de colchetes e parênteses.

```
import numpy as np

np.array([[3, 1], [2, 4]])

array([[3, 1],
       [2, 4]])
```

$$\begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix}$$

$a_{11} = 3$ (linha 1, coluna 1)

$a_{12} = 1$ (linha 1, coluna 2)

$a_{21} = 2$ (linha 2, coluna 1)

$a_{22} = 4$ (linha 2, coluna 2)

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

Matrizes e NumPy

Matriz 1: dimensão 2x2

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

$a_{11} = 3$ (linha 1, coluna 1)

$a_{12} = 1$ (linha 1, coluna 2)

$a_{21} = 2$ (linha 2, coluna 1)

$a_{22} = 4$ (linha 2, coluna 2)

```
import numpy as np

# Matriz 1: Dimensão 2x2
matriz1 = np.array([[3, 1], [2, 4]])

# Exibindo as matrizes
print("Matriz 1:")
print(matriz1)

matriz1

Matriz 1:
[[3 1]
 [2 4]]
array([[3, 1],
       [2, 4]])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

Matrizes e NumPy

Matriz 2: dimensão 3x3

$$\begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix}$$

$b_{11} = 5$ (linha 1, coluna 1)

$b_{12} = 2$ (linha 1, coluna 2)

$b_{13} = 7$ (linha 1, coluna 3)

$b_{21} = 8$ (linha 2, coluna 1)

$b_{22} = 9$ (linha 2, coluna 2)

$b_{23} = 3$ (linha 2, coluna 3)

$b_{31} = 4$ (linha 3, coluna 1)

$b_{32} = 6$ (linha 3, coluna 2)

$b_{33} = 0$ (linha 3, coluna 3)

```
import numpy as np

# Matriz 2: Dimensão 3x3
matriz2 = np.array([[5, 2, 7], [8, 9, 3], [4, 6, 0]])
```

```
print("Matriz 2:")
print(matriz2)
matriz2
```

```
Matriz 2:
[[5 2 7]
 [8 9 3]
 [4 6 0]]
array([[5, 2, 7],
       [8, 9, 3],
       [4, 6, 0]])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

Matrizes e NumPy

Matriz 3: dimensão 2x3

$$\begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \end{bmatrix}$$

$c_{11} = 2$ (linha 1, coluna 1)

$c_{12} = 6$ (linha 1, coluna 2)

$c_{13} = 3$ (linha 1, coluna 3)

$c_{21} = 0$ (linha 2, coluna 1)

$c_{22} = 1$ (linha 2, coluna 2)

$c_{23} = 4$ (linha 2, coluna 3)

```
import numpy as np

# Matriz 3: Dimensão 2x3
matriz3 = np.array([[2, 6, 3], [0, 1, 4]])

# Exibindo as matrizes
print("\nMatriz 3:")
print(matriz3)
matriz3
```

```
Matriz 3:
[[2 6 3]
 [0 1 4]]
array([[2, 6, 3],
       [0, 1, 4]])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

NumPy – Criando array

Vamos criar o array do NumPy de 1, 2 e 3 dimensões.

```
1 array_1D = np.array([1, 2, 3, 4])
2 array_1D
```

```
array([1, 2, 3, 4])
```

```
1 array_2D = np.array([[1, 2], [3, 4]])
2 array_2D
```

```
array([[1, 2],
       [3, 4]])
```

```
1 array_3D = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
2 array_3D
```

```
array([[[1, 2],
        [3, 4]],
       [[5, 6],
        [7, 8]]])
```

Command

`np.array([1,2,3])`



NumPy Array

1
2
3

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Reprodução – NUMPY, [s.d.]. Disponível em:
https://numpy.org/doc/stable/user/absolute_beginners.html. Acesso em: 4 jun. 2024.

Construindo o conceito

NumPy – Criando array

Existem algumas funções embutidas no NumPy para criar arrays.

- **np.zeros(shape)**: Cria um array com todos os elementos iguais a zero.

```
1 import numpy as np
2
3 # Utilizando np.zeros(shape) para criar uma matriz 2x3 preenchida com zeros
4 matriz_zeros = np.zeros((2, 3))
5 print("Matriz de zeros:")
6 print(matriz_zeros)
```

```
Matriz de zeros:
[[0. 0. 0.]
 [0. 0. 0.]]
```

```
1 # Formato do array
2 matriz_zeros
```

```
array([[0., 0., 0.],
       [0., 0., 0.]])
```



Dica

O parâmetro *shape* especifica a forma ou dimensão da matriz ou array que você deseja criar. Em outras palavras, *shape* determina o número de linhas e colunas (ou dimensões) da matriz ou array.

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

NumPy – Criando array

- `np.ones(shape)`: cria um array com todos os elementos iguais a um.

```
1 # Utilizando np.ones(shape) para criar uma matriz 3x4 preenchida com uns:
2
3 matriz_ones = np.ones((3, 4))
4 print("Matriz de uns:")
5 print(matriz_ones)
```

Matriz de uns:

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

```
1 # Formato do array
2 matriz_ones
```

```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

NumPy – Criando array

- `np.empty(shape)`: cria um array vazio (os valores serão gerados aleatoriamente).

```
1 # Utilizando np.empty(shape) para criar uma matriz 2x2 vazia:
2
3 matriz_vazia = np.empty((2, 2))
4 print("Matriz vazia:")
5 print(matriz_vazia) # os valores podem variar
```

```
Matriz vazia:
[[4.24399158e-314  8.48798317e-314]
 [1.27319747e-313  1.69759663e-313]]
```

```
1 # Formato do array
2 matriz_vazia
```

```
array([[4.24399158e-314,  8.48798317e-314],
       [1.27319747e-313,  1.69759663e-313]])
```



Tome nota

A função `np.empty(shape)` cria uma matriz sem inicializar seus valores, o que significa que os valores retornados podem ser qualquer coisa, incluindo valores que já existiam na memória antes de a matriz ser criada. Esses valores não são necessariamente zeros.

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

NumPy – Criando array

- `np.eye(N)`: cria uma matriz identidade NxN.

```
1 # Utilizando np.eye(N) para criar uma matriz identidade 3x3:
2
3 matriz_identidade = np.eye(3)
4 print("Matriz identidade:")
5 print(matriz_identidade)
```

Matriz identidade:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
1 # Formato do array
2 matriz_identidade
```

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

NumPy – Criando array

- `np.arange(início, fim, intervalo)`: cria um array com valores espaçados uniformemente.

```
1 # Utilizando np.arange(start, stop, step) para criar
2 # um array de números de 1 a 10, com intervalo de 2:
3
4 array_sequencial = np.arange(1, 11, 2)
5 print("Array sequencial:")
6 print(array_sequencial)
```

Array sequencial:
[1 3 5 7 9]

```
1 # Formato do array
2 array_sequencial
```

array([1, 3, 5, 7, 9])

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

NumPy – Inspeccionando array

Podemos inspecionar um array obtendo informações como seu tamanho, sua forma e o número de dimensões.

- **ndarray.ndim** informa o número de eixos ou dimensões do array. Em outras palavras, indica quantas dimensões o array possui.

```
1 # Array unidimensional
2 array_1d = np.array([1, 2, 3])
3 print("Número de dimensões do array unidimensional:", array_1d.ndim)
4
5 # Array bidimensional
6 array_2d = np.array([[1, 2, 3], [4, 5, 6]])
7 print("Número de dimensões do array bidimensional:", array_2d.ndim)
```

Número de dimensões do array unidimensional: 1

Número de dimensões do array bidimensional: 2

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

NumPy – Inspeccionando array

Podemos inspecionar um array obtendo informações como seu tamanho, sua forma e o número de dimensões.

- **ndarray.size** informa o número total de elementos do array. Este é o produto dos elementos da forma (*shape*) do array.

```
1 # Array bidimensional
2 array_2d = np.array([[1, 2, 3], [4, 5, 6]])
3 print("Número total de elementos do array bidimensional:", array_2d.size)
```

Número total de elementos do array bidimensional: 6

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

NumPy – Inspeccionando array

Podemos inspecionar um array obtendo informações como seu tamanho, sua forma e o número de dimensões.

- **ndarray.shape** exibe uma tupla de inteiros que indicam o número de elementos armazenados ao longo de cada dimensão do array.

```
1 # Array bidimensional
2 array_2d = np.array([[1, 2, 3], [4, 5, 6]])
3 print("Forma (shape) do array bidimensional:", array_2d.shape)
```

Forma (shape) do array bidimensional: (2, 3)

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.



© Getty Images

O que nós
**aprendemos
hoje?**

Então ficamos assim...

- 1** Vimos a relação entre as matrizes de álgebra linear com o array do NumPy.
- 2** Aprendemos diversas formas de criar um array no NumPy.
- 3** Inspecionamos o array quanto à forma, tamanho e número de dimensões.

Saiba mais

Quer turbinar seus projetos em Python e realizar cálculos complexos com mais rapidez e eficiência?

O NumPy é a solução perfeita para você!

ALMEIDA, M. *Numpy*: trabalhando computação científica com Python. Alura, 13 mar. 2023. Disponível em: <https://www.alura.com.br/artigos/numpy-computacao-cientifica-com-python>.

Acesso em: 4 jun. 2024.

Referências da aula

MCKINNEY, W. *Python para análise de dados: tratamento de dados com Pandas, NumPy & Jupyter*. São Paulo: Novatec, 2023.

NUMPY. *Página inicial*, [s.d.]. Disponível em: <https://numpy.org/pt/>. Acesso em: 4 jun. 2024.

Identidade visual: imagens © Getty Images.

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**